

Factorization

You are given positive integers N and M .

How many sequences a of length N consisting of positive integers satisfy

$$a_1 \times a_2 \times \dots \times a_N = M?$$

Find the count modulo $10^9 + 7$.

Constraints

All values in input are integers.

$$1 \leq N \leq 10^5$$

$$1 \leq M \leq 10^9$$

Multiplication problems become addition problems after prime factorization.

Let:

$$M = p_1^{e_1} \cdot p_2^{e_2} \cdot \dots \cdot p_k^{e_k}$$

Example:

$$M = 12 = 2^2 \times 3^1$$

Now look at the condition:

$$a_1 \cdot a_2 \dots a_n = M$$

This is equivalent to saying:

For each prime p_j :

$$(\text{exponent of } p_j \text{ in } a_1) + \dots + (\text{exponent of } p_j \text{ in } a_n) = e_j$$

So the problem splits independently per prime.

This independence is the most important insight.

Reduce to a Stars and Bars problem (per prime)

Fix one prime p with exponent e .

Define:

$$x_1 + x_2 + \dots + x_n = e$$

where:

$$x_i \geq 0$$

$$x_i = \text{exponent of } p \text{ in } a_i$$

This is exactly the classic Stars and Bars problem.

Stars and Bars result

The number of non-negative integer solutions to:

$$x_1 + x_2 + \dots + x_n = e$$

is:

$$\binom{e + N - 1}{N - 1}$$

So:

For a single prime p^e , the number of ways to distribute its exponent among the N positions is

$$\binom{e + N - 1}{N - 1}$$

Combine all primes (why we multiply)

Each prime's exponent distribution is independent of the others.

- Choices for prime 2 do not affect choices for prime 3
- For each valid distribution of prime 2, we can choose any valid distribution of prime 3, etc.

By the product rule:

$$\text{Answer} = \prod_{j=1}^k \binom{e_j + N - 1}{N - 1}$$

Product constraint $\rightarrow$ prime exponents

Prime exponents $\rightarrow$ sum constraint

Sum constraint with non-negatives $\rightarrow$ Stars and Bars

Code

```
#include <bits/stdc++.h>
using namespace std;

static const long long MOD = 1000000007;
static const int MAXN = 200000; // enough for N + max exponent

long long fact[MAXN + 1], invfact[MAXN + 1];

long long modpow(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++)
        fact[i] = fact[i - 1] * i % MOD;

    invfact[MAXN] = modpow(fact[MAXN], MOD - 2);
    for (int i = MAXN; i > 0; i--)
        invfact[i - 1] = invfact[i] * i % MOD;
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute();

    long long N, M;
    cin >> N >> M;

    long long ans = 1;

    for (long long p = 2; p * p <= M; p++) {
        if (M % p == 0) {
            int cnt = 0;
            while (M % p == 0) {
                M /= p;
                cnt++;
            }
            ans = ans * nCr(cnt + N - 1, N - 1) % MOD;
        }
    }

    // If M is still > 1, it's a prime
    if (M > 1) {
        ans = ans * nCr(1 + N - 1, N - 1) % MOD;
    }

    cout << ans << "\n";
    return 0;
}
```